AutoDD: Learning to Select Drift Detectors for Evolving Data Streams

Anonymous

***Abstract*—Concept drift, the change in the distribution of streaming data over time, significantly impacts the performance of machine learning models. While numerous drift detection methods have been proposed, selecting and configuring the most suitable detector for a specific data stream remains a challenging task. Existing AutoML frameworks for streaming data that automate model selection rely on fixed drift detection strategies, leaving a critical gap in automation. This paper introduces *AutoDD* (Automated Drift Detector), a novel framework that jointly automates detector selection and hyperparameter configuration using a hierarchical learning approach. AutoDD extracts comprehensive online meta-features and employs a two-level hierarchical selection algorithm to adaptively choose both the detector and its configuration. Experiments on ten synthetic and real-world datasets demonstrate that AutoDD consistently outperforms state-of-the-art detectors while maintaining computational efficiency.**

***Index Terms*—concept drift detection, algorithm selection, contextual bandits, online meta-learning, data streams**

I. Introduction

Concept drift, where the statistical properties of streaming data evolve over time, poses a significant challenge in data stream mining [1], [2], incremental learning [3], and non-stationary learning [4]. As data distributions change, machine learning models experience degraded performance, necessitating timely detection and adaptation. This phenomenon manifests in various forms, including sudden (abrupt) drift, gradual drift, incremental drift, and recurring drift, as illustrated in Figure 1. For instance, an electricity price prediction model may encounter sudden drift when market regulations change abruptly, experience gradual drift as consumer behaviour shifts over seasons, or observe recurring drift driven by daily demand cycles [4].

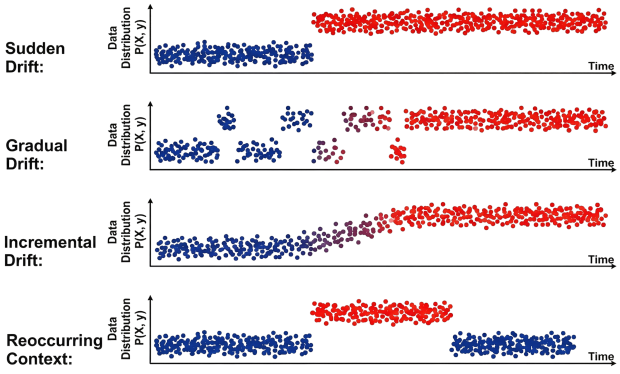


Fig. 1: Types of concept drifts.

Over the past two decades, numerous drift detection methods have been developed to address this challenge, spanning error rate-based, distribution-based, uncertainty-based, and meta-learning paradigms [1]. Yet no single detector performs best across all conditions, because each embeds assumptions about how drift manifests. As a result, a detector well suited to abrupt changes may respond slowly to gradual ones, and the most appropriate choice can shift as the stream itself evolves. However, a fundamental challenge remains unresolved in systematically selecting and configuring the most appropriate drift detector for a given data stream.

Automated Machine Learning (AutoML) has revolutionised the field by addressing similar selection challenges in batch learning scenarios [5]. Recent advancements have extended AutoML to streaming data, enabling automatic selection and configuration for supervised learning tasks [6], [7]. However, these systems treat drift detection as a fixed component, requiring manual specification and leaving a critical gap in the automation framework.

Automating this choice online is challenging because drift labels are unavailable as the stream arrives [1], [4], so detector quality must be inferred from a proxy reward derived from downstream model behaviour rather than from ground-truth change points [8], [9], and because the joint space of detectors and their configurations is too large to search as a single flat action set [5]. Unlike detector ensembles that run and combine many detectors at once [10], [11], or meta-learning methods that learn a fixed mapping offline [9], an effective solution must select and configure one detector online and continually refine this choice as the stream evolves. To bridge this gap, we introduce *AutoDD* (Automated Drift Detector), a modular framework that automates drift detector selection and configuration using a hierarchical learning approach, compatible with any model or AutoML pipeline. AutoDD makes three main contributions.

- We propose *AutoDD*, a novel framework for automated drift detector selection and configuration in evolving data streams.
- AutoDD formalises the problem of joint detector and hyperparameter selection and solves it via a two-level hierarchical bandit using online meta-features, LinUCB [12] for detector selection, and Thompson Sampling [13] for hyperparameter configuration.
- A comprehensive experimental study demonstrates AutoDD consistently outperforms state-of-the-art detectors across ten benchmarks while maintaining computational efficiency.

II. Prior Work

Drift detection methods can be broadly categorised into three paradigms. **Error rate-based methods** monitor fluctuations in classifier error rates, triggering alarms when changes are statistically significant. DDM [14] pioneered the warning-drift framework by monitoring prediction accuracy under binomial distribution assumptions, followed by EDDM [15] which focuses on distances between errors to detect gradual drift, HDDM [16] which applies Hoeffding's inequality with exponentially weighted moving averages, and ADWIN [8] which maintains adaptive sliding windows that automatically shrink when distribution changes are detected. Page-Hinkley [17] applies cumulative sum control charts to detect mean shifts in sequential data. Recent advances include MDDM [18], which uses McDiarmid's inequality for shorter detection delays with tighter concentration bounds, and ADDM [19], which models error rates as SETAR time series [20] to achieve lower false alarm rates. **Distribution-based methods** compare data distributions across windows by quantifying dissimilarity between new and historical data distributions [21], including approaches based on information theory [22], competence models [23], and the Kolmogorov-Smirnov test (KSWIN) [24] which detects changes in univariate feature distributions. KSWIN is a recent representative of the statistical two-sample test family, which also includes earlier detectors such as IKS [25] and WRS [10]. **Uncertainty and learning-based methods** address limitations of error-rate monitoring: PUDD [26] leverages prediction confidence for earlier detection by monitoring changes in prediction uncertainty distributions, while Meta-ADD [9] employs prototypical networks to classify drift types via meta-learning on diverse drift scenarios.

Recent developments in automated algorithm selection and hyperparameter optimisation (CASH) [5] have been extended to streaming scenarios [27]. However, these streaming AutoML systems automate supervised task selection while assuming a fixed, manually specified drift detector, so the detector itself is never selected or configured automatically. AutoDD addresses this gap by bringing the detector into the automation loop, learning online which detector and configuration best suit the current stream.

III. Problem Formulation

We consider a supervised data stream in which instances arrive sequentially and potentially without end. At each time step t, the stream produces an instance $(\mathbf{x}_t, y_t)$, where $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ is the feature vector and y_t is its label, drawn from a joint distribution $P_t(\mathbf{x}, y)$. The stream $\mathcal{S} = \{(\mathbf{x}_t, y_t)\}_{t=1}^{\infty}$ exhibits concept drift when this distribution changes over time, that is, when

$$\exists t, t' : P_t(\mathbf{x}, y) \neq P_{t'}(\mathbf{x}, y). \tag{1}$$

Given a detector portfolio $\mathcal{D} = \{D_1, \ldots, D_K\}$ where each D_i has configuration space $\mathcal{H}_i$, AutoDD addresses the streaming detector selection and configuration problem. Inspired by the CASH formulation [5], we jointly optimise the detector choice and its hyperparameters:

$$(D_t^*, \boldsymbol{\eta}_t^*) = \arg\max_{D_i \in \mathcal{D}, \boldsymbol{\eta} \in \mathcal{H}_i} \mathbb{E}[\text{Perf}(D_i(\boldsymbol{\eta}), \mathcal{S}_t) \mid \boldsymbol{\phi}_t] \tag{2}$$

where $\boldsymbol{\phi}_t$ is the stream context, $\mathcal{S}_t$ denotes the stream segment at time t, $\mathbb{E}[\cdot]$ is the expectation operator, and $\text{Perf}(\cdot)$ denotes the downstream model performance.

IV. Automated Drift Detector

AutoDD employs an adaptive, closed-loop architecture to dynamically select and configure drift detectors. As shown in Figure 2, this architecture operates through several key steps. First, the system initialises the detector portfolio with standard hyperparameters and sets up the learning parameters. During online operation, meta-features characterising stream velocity, stationarity, and temporal patterns are continuously extracted from the incoming data stream. These meta-features serve as context for the LinUCB agent, which selects a detector from the portfolio. Thompson Sampling then chooses a hyperparameter configuration for the selected detector. The configured detector monitors the stream and triggers model adaptation upon detecting drift. Finally, performance feedback is computed based on detection outcomes, which updates both the LinUCB and Thompson Sampling components to refine future selections. This adaptive learning cycle enables AutoDD to improve its detector selections over time.

A. Algorithm Overview

AutoDD operates over sliding windows of size w. When each window fills with incoming instances, the framework executes a complete selection cycle, beginning with meta-feature extraction.

Meta-Feature Extraction: The meta-feature set comprises 15 features in three categories. *Statistical* features capture velocity, dimensionality, sparsity, noise level, and class imbalance; *stationarity* features capture error volatility, KL divergence, autocorrelation, mean shift, and variance shift; and *temporal* features capture trend, periodicity, change frequency, change magnitude, and recurrence score. All features are computed over a sliding window of size w using online statistics and normalised to $[0, 1]$ via running min-max normalisation to ensure scale consistency. The resulting context vector $\boldsymbol{\phi}_t \in \mathbb{R}^m$ with $m = 15$ is computed incrementally, supporting efficient per-instance updates.

Detector Portfolio: AutoDD maintains a diverse portfolio of $K = 7$ drift detectors spanning window-based, error-based, and distribution-based paradigms. Table I summarises each detector with its hyperparameters, valid ranges, sampling scales, and default values from the literature. For each detector i, we maintain C hyperparameter configurations denoted $\boldsymbol{\eta}_{i,c}$ for $c \in [C]$, the first is always the literature default, while the remaining $C - 1$ are sampled from the specified ranges using the appropriate logarithmic, linear, or integer scale.

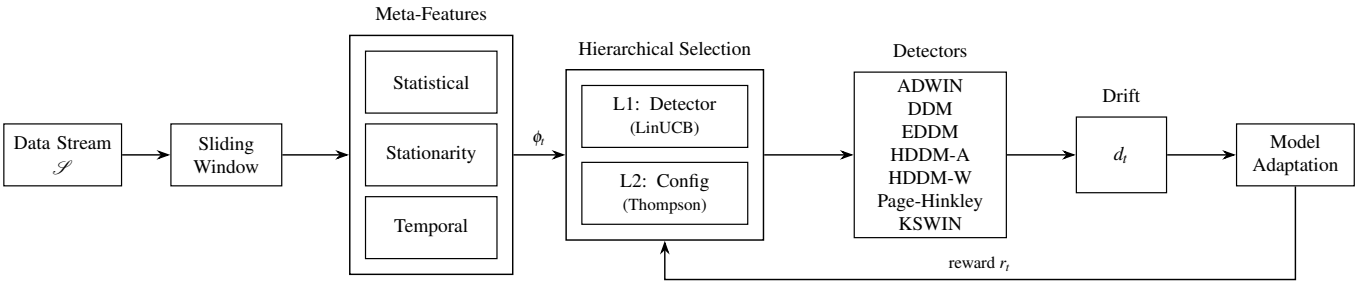


Fig. 2: Overview of the AutoDD framework. The incoming data stream $\mathcal{S}$ is buffered into a sliding window, from which statistical, stationarity, and temporal meta-features form the context $\boldsymbol{\phi}_t$. Hierarchical selection then chooses a detector with LinUCB (Level 1) and a hyperparameter configuration with Thompson Sampling (Level 2) from the detector portfolio. The configured detector emits a drift signal d_t that triggers model adaptation, and the resulting reward r_t is fed back to update both selection levels.

TABLE I: Detector portfolio with hyperparameter configurations.

Detector	Type	Parameter	Range	Scale	Default
ADWIN	Window-based	δ_{adwin}	[0.0001, 0.1]	log	0.002
Page-Hinkley	Window-based	θ_{ph}	[10, 100]	int	50
		δ_{ph}	[0.001, 0.01]	log	0.005
DDM	Error-based	w_{thresh}	[1.5, 3.0]	linear	2.0
		d_{thresh}	[2.0, 4.0]	linear	3.0
EDDM	Error-based	α_{eddm}	[0.90, 0.99]	linear	0.95
		β_{eddm}	[0.85, 0.95]	linear	0.90
HDDM-A	Error-based	δ_{hddm}	[0.0001, 0.01]	log	0.001
HDDM-W	Error-based	λ_{hddm}	[0.01, 0.1]	log	0.05
KSWIN	Distribution-based	α_{ks}	[0.001, 0.05]	log	0.005
		w_{ks}	[50, 500]	int	100

B. Hierarchical Learning Approach

AutoDD uses a two-level hierarchical selection strategy that jointly adapts detector choice and hyperparameter configuration while maintaining computational efficiency. This decomposition avoids directly treating all $K \times C$ detector-configuration pairs as a single flat action space.

Level 1 - Detector Selection via LinUCB: We formulate detector selection as a contextual multi-armed bandit problem where context is $\boldsymbol{\phi}_t$ (meta-features), arms are K detectors, and reward is detection performance. LinUCB [12] assumes linear reward:

$$r_{i,t} = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* + \epsilon_t \tag{3}$$

where $\boldsymbol{\theta}_i^* \in \mathbb{R}^m$ is the unknown parameter vector for detector i, with m being the meta-feature dimensionality. The algorithm selects the detector with the highest upper confidence bound:

$$i_t = \arg\max_{i \in [K]} \left(\boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha_{\text{ucb}} \sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \boldsymbol{\phi}_t} \right) \tag{4}$$

where $\hat{\boldsymbol{\theta}}_i = \mathbf{A}_i^{-1}\mathbf{b}_i$ is the ridge regression estimate. Here, $\mathbf{A}_i = \lambda_{\text{reg}}\mathbf{I} + \sum_{s:i_s=i} \boldsymbol{\phi}_s \boldsymbol{\phi}_s^\top \in \mathbb{R}^{m\times m}$ is the regularised design matrix with regularisation parameter $\lambda_{\text{reg}} > 0$, and $\mathbf{b}_i = \sum_{s:i_s=i} r_s \boldsymbol{\phi}_s \in \mathbb{R}^m$ is the cumulative reward-weighted context vector for detector i. The parameter $\alpha_{\text{ucb}} > 0$ controls exploration-exploitation balance.

Level 2 - Hyperparameter Selection via Thompson Sampling: Given the selected detector D_{i_t}, we adaptively configure its hyperparameters using Thompson Sampling [13]. For each configuration $c \in [C] = \{1, \ldots, C\}$ of detector i, we maintain Beta distribution parameters $(a_{i,c}, b_{i,c})$ initialised to $(1, 1)$. At each window, we sample:

$$\psi_{i_t,c} \sim \text{Beta}(a_{i_t,c}, b_{i_t,c}) \quad \forall c \in [C] \tag{5}$$

and select configuration:

$$c_t = \arg\max_c \psi_{i_t,c} \tag{6}$$

After observing reward r_t, we update:

$$a_{i_t,c_t} \leftarrow a_{i_t,c_t} + \tilde{r}_t \tag{7}$$

$$b_{i_t,c_t} \leftarrow b_{i_t,c_t} + (1 - \tilde{r}_t) \tag{8}$$

where $\tilde{r}_t = (r_t + 1)/2$ normalises the reward to $[0, 1]$.

C. Evaluation and Adaptation

Evaluating detector performance without ground-truth drift labels is challenging. Following standard practice, we use model accuracy drop as a proxy metric. The reward at time t is:

$$r_t = \begin{cases} 0.5 + \min(\Delta a_t \cdot 2, 0.5) & d_t = 1, \Delta a_t > \tau_{\text{sig}} \\ -0.3 & d_t = 1, \Delta a_t \le \tau_{\text{sig}} \\ -0.5 - \min(\Delta a_t, 0.5) & d_t = 0, \Delta a_t > \tau_{\text{sig}} \\ 0.1 & d_t = 0, \Delta a_t \le \tau_{\text{sig}} \end{cases} \tag{9}$$

where $d_t \in \{0, 1\}$ indicates drift detection at time t, $\Delta a_t = a_{t-w} - a_t$ is the accuracy drop between consecutive windows, and $\tau_{\text{sig}} = 0.05$ is the significance threshold. The reward function penalizes both false positives and false negatives while rewarding correct drift detections and stable periods. We note that an accuracy-based proxy couples detector quality with the downstream learner, label noise, and the evaluation window. Two design choices limit the resulting variance: the drop Δa_t is measured over a window of w instances rather than per-instance, smoothing transient noise, and the threshold τ_{sig} suppresses rewards from sub-significant fluctuations so that noise-driven dips do not reinforce spurious detections. Because the reward enters the bandit update only as a bounded scalar in $[-1, 1]$, occasional misattributed rewards perturb the posteriors gradually rather than abruptly. The regret analysis below is therefore stated for this bounded proxy reward;

interpreting proxy regret as true detector quality requires the proxy-fidelity assumption introduced below.

Upon drift detection ($d_t = 1$), the model resets to its initial state $\mathcal{M}_0$:

$$\mathcal{M}_t \leftarrow \mathcal{M}_0 \tag{10}$$

The model then resumes incremental learning from subsequent instances:

$$\mathcal{M}_{t+s} \leftarrow \text{Update}(\mathcal{M}_{t+s-1}, (\mathbf{x}_{t+s}, y_{t+s})) \quad \forall s > 0 \tag{11}$$

The complete procedure is formalized in Algorithm 1, which presents the per-window form with a single joint update per window; the deployed implementation additionally updates Level 1 at each detected drift while updating Level 2 only at window boundaries.

Algorithm 1: AutoDD

Input: Stream $\mathcal{S}$, portfolio $\mathcal{D}$, window w; exploration α_{ucb}, regularisation λ_{reg}, configs per detector C
Output: Drift signals $\{d_t\}_{t=1}^{\infty}$

Initialise $\mathbf{A}_i \leftarrow \lambda_{\text{reg}}\mathbf{I}_m$, $\mathbf{b}_i \leftarrow \mathbf{0}$ for all $i \in [K]$;
Initialise $a_{i,c} \leftarrow 1$, $b_{i,c} \leftarrow 1$ for all $i \in [K], c \in [C]$;
Initialise model $\mathcal{M}_0$, sliding window $W \leftarrow \emptyset$;
foreach *instance* $(\mathbf{x}_t, y_t) \in \mathcal{S}$ **do**
  $W \leftarrow W \cup \{(\mathbf{x}_t, y_t)\}$;
  if $|W| = w$ **then**
    $\boldsymbol{\phi}_t \leftarrow \text{EXTRACT}(W)$; `// meta-features`
    for $i = 1$ **to** K **do**
      $\text{UCB}_i \leftarrow \boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha_{\text{ucb}} \sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \boldsymbol{\phi}_t}$;
       // $\hat{\boldsymbol{\theta}}_i = \mathbf{A}_i^{-1}\mathbf{b}_i$
    $i_t \leftarrow \arg\max_i \text{UCB}_i$; `// select detector`
    for $c = 1$ **to** C **do**
      Sample $\psi_c \sim \text{Beta}(a_{i_t,c}, b_{i_t,c})$;
    $c_t \leftarrow \arg\max_c \psi_c$; `// select configuration`
    $d_t \leftarrow D_{i_t}(\boldsymbol{\eta}_{i_t,c_t}).\text{DETECT}(W)$;
    if $d_t = 1$ **then**
      $\mathcal{M}_t \leftarrow \mathcal{M}_0$; `// reset model`
    Compute reward r_t using Eq. (9);
    $\mathbf{A}_{i_t} \leftarrow \mathbf{A}_{i_t} + \boldsymbol{\phi}_t\boldsymbol{\phi}_t^\top$; $\mathbf{b}_{i_t} \leftarrow \mathbf{b}_{i_t} + r_t\boldsymbol{\phi}_t$;
    $\tilde{r}_t \leftarrow (r_t + 1)/2$;
    $a_{i_t,c_t} \leftarrow a_{i_t,c_t} + \tilde{r}_t$; $b_{i_t,c_t} \leftarrow b_{i_t,c_t} + (1 - \tilde{r}_t)$;
    $W \leftarrow \emptyset$;

D. Computational Complexity

AutoDD has per-window complexity of $O(w\, d_{\text{in}} + Km^2)$ for the incrementally maintained meta-features used in our implementation, where w is the AutoDD window size, d_{in} is the input feature dimensionality, K is the number of detectors, and $m = 15$ is the meta-feature count. The main costs are meta-feature extraction ($O(w\, d_{\text{in}})$) and LinUCB matrix operations ($O(Km^2)$). Space complexity is $O(Km^2 + w\, d_{\text{in}})$ for storing K covariance matrices of size $m \times m$ and the sliding window buffer.

E. Theory

Building on the selection objective formalised in Section III, we provide proxy-regret bounds for the selection mechanism. The analysis treats the two levels as idealised contextual-bandit sub-problems, and where the analysis simplifies the deployed system is noted throughout.

Assumptions. We adopt three standard assumptions from the contextual bandit literature [28], [29], together with a fidelity condition (A4) that links our accuracy-based proxy reward to true detection quality:

- **(A1) Bounded features:** $\|\boldsymbol{\phi}_t\|_2 \le L$ for all t, satisfied by $[0,1]$ normalisation with $L = \sqrt{m}$.
- **(A2) Linear reward:** For each detector i, there exists $\boldsymbol{\theta}_i^* \in \mathbb{R}^m$ with $\|\boldsymbol{\theta}_i^*\|_2 \le S$ such that $\mathbb{E}[r_{i,t} \mid \boldsymbol{\phi}_t] = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^*$, where $S > 0$ is a constant upper bound on the parameter norms.
- **(A3) Sub-Gaussian noise:** Reward noise is σ_{noise}-sub-Gaussian, satisfied with $\sigma_{\text{noise}} = 1$ for bounded rewards in $[-1, 1]$.
- **(A4) Proxy fidelity:** The windowed accuracy-drop reward (Eq. (9)) is, in expectation, monotone in true detection quality, so that a high proxy-reward detector-configuration pair is also high quality under the objective in Eq. (2). The bounds below are stated with respect to this proxy objective.

A2 treats the windowed reward as linear in the meta-features in expectation, an approximation examined empirically through the selection-accuracy trend in Appendix Table XI. The two results below analyse idealised sub-problems and are best read as surrogate analyses rather than guarantees for the coupled, deployed system.

Remark (stationarity under drift). Although P_t is non-stationary, A2 requires only that the conditional reward map $\boldsymbol{\phi}_t \mapsto \mathbb{E}[r_{i,t} \mid \boldsymbol{\phi}_t]$ be stationary, not the marginal P_t. Changes in P_t are represented through the context rather than through time-varying $\boldsymbol{\theta}_i^*$, and the monotonic rise of proxy-oracle selection accuracy in Appendix Table XI is consistent with this approximation on the streams we study.

The following result gives the detector-selection proxy regret for LinUCB under the stationary linear abstraction.

Theorem 1 (Level 1 Regret). Fix each detector's configuration, so that A1–A3 hold for the resulting stationary per-detector reward. With time horizon T, confidence parameter $\delta_{\text{conf}} \in (0, 1)$, and the standard OFUL exploration radius

$$\alpha_{\text{ucb}} = \sigma_{\text{noise}} \sqrt{m \log(KT/\delta_{\text{conf}})} + \sqrt{\lambda_{\text{reg}}}\, S, \tag{12}$$

disjoint-arm LinUCB detector selection achieves, with probability at least $1 - \delta_{\text{conf}}$,

$$R_1(T) = \textstyle\sum_{t=1}^{T} \left(\max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^* \right) = \tilde{O}(m\sqrt{KT}). \tag{13}$$

Proof Sketch. We analyse the disjoint-arm LinUCB algorithm [12] using self-normalised confidence bounds [28]. The self-normalised martingale inequality places each $\boldsymbol{\theta}_i^*$, with probability $1 - \delta_{\text{conf}}$, in the confidence ellipsoid

$$\|\boldsymbol{\theta}_i^* - \hat{\boldsymbol{\theta}}_i\|_{\mathbf{A}_i} \leq \alpha_{\text{ucb}}. \tag{14}$$

On this event the UCB rule bounds the instantaneous regret as

$$\max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^* \;\leq\; 2\alpha_{\text{ucb}} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}}. \tag{15}$$

Summing over t, applying the elliptical potential lemma [30] to each of the K per-detector estimators, and combining the potentials by Cauchy–Schwarz contributes the $\sqrt{K}$ factor and, with the radius $\alpha_{\text{ucb}} = \tilde{O}(\sqrt{m})$, gives $R_1(T) = \tilde{O}(m\sqrt{KT})$. □

Proposition 2 (Level 2 Proxy Regret). Fix a detector i and suppose its C configurations have stationary mean rewards. For Beta–Bernoulli Thompson Sampling under the standard Bernoulli-sampling reduction for bounded rewards, with $N_i(T)$ the number of rounds detector i is selected, the expected regret over those rounds satisfies:

$$R_2(T) = \sum_{i=1}^{K} O(\sqrt{C \cdot N_i(T) \log N_i(T)}) \tag{16}$$

Proof Sketch. Conditioned on detector i being played, hyperparameter selection is a C-armed bandit with rewards normalised to $\tilde{r}_t = (r_t + 1)/2 \in [0, 1]$. Under the Bernoulli-sampling reduction with Beta posteriors, Thompson Sampling attains expected regret $O(\sqrt{CN_i(T) \log N_i(T)})$ over the $N_i(T)$ rounds detector i is played [31]; summing over the K detectors with $\sum_i N_i(T) = T$ gives the bound. The deployed fractional Beta update (Eq. (7)) replaces each sampled Bernoulli outcome by its conditional expectation; the bound is therefore a surrogate for the idealised stationary-Bernoulli problem, not a guarantee for the implemented update. □

These two results bound idealised sub-problems in isolation and do *not* compose into a guarantee against a joint detector–configuration oracle. If the surrogate objectives are treated as separable, telescoping through the best configuration of the selected detector and bounding the configuration term by Cauchy–Schwarz gives the indicative combined rate

$$R(T) \;\lesssim\; \tilde{O}(m\sqrt{KT}) + O(\sqrt{CKT \log T}), \tag{17}$$

which we report only as a heuristic surrogate. The empirical convergence behaviour of the deployed system is examined separately in Section VI.

V. Experiments

We evaluate AutoDD on ten benchmark datasets comprising five synthetic and five real-world streams, with characteristics detailed in Table II. To isolate drift detection performance from model selection effects, all methods use two fixed base models: GaussianNB and Hoeffding Adaptive Tree (HAT) [32]. These two learners represent fundamentally different model families (probabilistic and tree-based) and are the standard benchmark base learners used in the data stream mining literature [2], [10]. We compare AutoDD against twelve baselines: a no-detector baseline and seven portfolio detectors (ADWIN, DDM, EDDM, HDDM-A, HDDM-W, Page-Hinkley, KSWIN). We also include four state-of-the-art methods: MDDM [18], ADDM [19], PUDD [26], and Meta-ADD [9]. All detectors use recommended hyperparameter settings from the literature. The portfolio includes one recent representative per detector family rather than every variant; because AutoDD's selection mechanism is detector-agnostic, a larger portfolio extends only the action set, not the method.

TABLE II: Dataset characteristics (Multiple* = Sudden, Gradual, Incremental, and Recurring drift types)

Dataset	Type	Instances	Features	Drift Type
SEA	Synthetic	500,000	3	Multiple*
HYP	Synthetic	500,000	10	Multiple*
AGR	Synthetic	100,000	9	Multiple*
RBF	Synthetic	200,000	10	Multiple*
RTG	Synthetic	100,000	30	Multiple*
Elec	Real-world	45,312	8	Unknown
Weather	Real-world	18,159	8	Unknown
Spam	Real-world	4,601	57	Unknown
Airlines	Real-world	539,383	7	Unknown
Poker	Real-world	829,201	10	Unknown

We measure prequential accuracy as the percentage of correct predictions using test-then-train evaluation [2]. For synthetic datasets with known drift points, we report F1-score computed by matching detected drifts to ground truth using a tolerance window of $\tau_{\text{tol}} = 100$ instances following standard practice [8]. For real-world datasets, we report the number of detected drifts (DN). These metrics are complementary by design, as prequential accuracy captures the downstream task impact of a detector's adaptation decisions, whereas the drift-detection F1-score scores precision and recall against ground-truth drift points and thus credits timely, correct detections while penalizing both missed drifts and spurious alarms that accuracy alone would not separate. On real-world streams the drift points are unknown, so F1 cannot be computed and we instead report DN alongside accuracy; following the benchmarking guidance [33], we interpret DN relative to the consensus detection range across methods rather than as an absolute target, treating values far above or below this range as over- and under-detection respectively. AutoDD uses all 15 meta-features with window size $w = 100$, exploration parameter $\alpha_{\text{ucb}} = 1.0$, regularisation parameter $\lambda_{\text{reg}} = 1.0$, and $C = 5$ hyperparameter configurations per detector. Results are averaged over 10 runs with random seeds uniformly sampled from the range [1, 100]. Implementation uses Python with River [34] on a MacBook Pro with Apple M2 Pro and 16GB RAM.[1]

VI. Results

We conduct a comprehensive evaluation of AutoDD against classical detectors and state-of-the-art methods across both base models.

[1]Code and datasets: https://anonymous.4open.science/r/AutoDD

TABLE III: Synthetic datasets – GaussianNB. Rank in (·); **bold** = best, underline = second-best per column.

Method	SEA		HYP		AGR		RBF		RTG		Mean	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
No Detector	83.8 (11)	0.40 (10)	76.4 (11)	0.34 (8)	**78.8 (1)**	0.41 (10)	50.4 (13)	0.09 (10)	58.2 (11)	0.10 (7)	69.5 (12)	0.27 (10)
ADWIN	84.0 (10)	0.61 (9)	81.9 (9)	0.00 (9)	71.5 (10)	0.51 (7)	69.6 (10)	0.00 (11)	64.5 (9)	0.00 (8)	74.3 (10)	0.22 (13)
DDM	83.8 (11)	0.00 (12)	86.1 (7)	0.63 (7)	71.6 (9)	0.49 (9)	71.3 (4)	0.64 (4)	64.4 (10)	0.00 (8)	75.4 (7)	0.35 (9)
EDDM	84.3 (9)	0.40 (10)	85.7 (8)	0.63 (7)	65.3 (11)	0.24 (11)	67.7 (12)	0.38 (9)	65.0 (8)	0.40 (5)	73.6 (11)	0.41 (8)
HDDM-A	84.6 (7)	0.69 (6)	81.5 (10)	0.00 (9)	72.2 (8)	0.63 (6)	69.7 (9)	0.00 (11)	64.4 (10)	0.00 (8)	74.5 (9)	0.26 (11)
HDDM-W	84.4 (8)	0.66 (7)	86.6 (6)	0.71 (4)	73.1 (6)	0.67 (5)	70.1 (6)	0.51 (8)	65.8 (6)	0.56 (4)	76.0 (6)	0.62 (6)
Page-Hinkley	84.7 (6)	0.70 (5)	81.5 (10)	0.00 (9)	71.5 (10)	0.49 (9)	69.4 (11)	0.00 (11)	64.5 (9)	0.00 (8)	74.3 (10)	0.24 (12)
KSWIN	83.7 (12)	0.35 (11)	85.7 (8)	0.65 (6)	71.5 (10)	0.50 (8)	70.0 (7)	0.61 (6)	65.2 (7)	0.39 (6)	75.2 (8)	0.50 (7)
MDDM	86.7 (4)	0.65 (8)	87.2 (5)	0.68 (5)	72.3 (7)	0.71 (4)	69.8 (8)	0.56 (7)	66.1 (5)	0.56 (4)	76.4 (5)	0.63 (5)
ADDM	87.7 (3)	0.83 (3)	88.7 (4)	0.73 (3)	73.6 (5)	0.67 (5)	70.9 (5)	0.62 (5)	67.6 (4)	0.71 (3)	77.7 (4)	0.71 (4)
PUDD	86.2 (5)	0.81 (4)	89.5 (3)	0.84 (2)	73.7 (4)	0.72 (3)	73.4 (3)	0.72 (2)	67.8 (3)	0.71 (3)	78.1 (3)	0.76 (3)
Meta-ADD	88.9 (2)	0.87 (2)	90.8 (2)	**0.91 (1)**	74.1 (3)	0.79 (2)	**78.0 (1)**	0.70 (3)	**72.9 (1)**	0.76 (2)	80.9 (2)	0.81 (2)
AutoDD	**93.1 (1)**	**0.90 (1)**	**94.6 (1)**	**0.91 (1)**	78.5 (2)	**0.84 (1)**	74.3 (2)	**0.81 (1)**	69.2 (2)	**0.84 (1)**	**81.9 (1)**	**0.86 (1)**

TABLE IV: Synthetic datasets – HAT. Rank in (·); **bold** = best, underline = second-best per column.

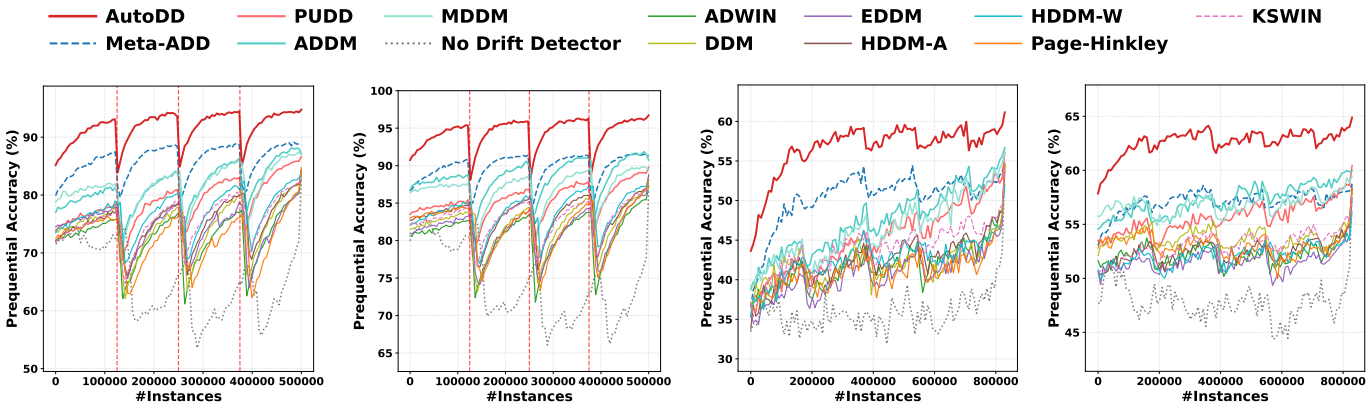
Method	SEA		HYP		AGR		RBF		RTG		Mean	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
No Detector	86.2 (10)	0.42 (9)	79.1 (13)	0.36 (10)	81.3 (2)	0.43 (11)	53.1 (12)	0.11 (8)	60.8 (13)	0.12 (10)	72.1 (13)	0.29 (11)
ADWIN	88.1 (8)	0.64 (7)	85.5 (12)	0.06 (11)	74.6 (10)	0.55 (8)	73.3 (10)	0.07 (9)	68.3 (11)	0.11 (11)	77.9 (11)	0.29 (11)
DDM	88.0 (9)	0.10 (10)	89.0 (7)	0.67 (8)	74.7 (9)	0.52 (10)	74.6 (5)	0.69 (3)	68.1 (12)	0.13 (9)	78.9 (6)	0.42 (9)
EDDM	88.0 (9)	0.46 (8)	88.6 (8)	0.68 (7)	69.2 (12)	0.29 (12)	71.8 (11)	0.42 (7)	68.8 (9)	0.47 (7)	77.3 (12)	0.46 (8)
HDDM-A	88.3 (6)	0.64 (7)	86.8 (10)	0.06 (11)	75.1 (7)	0.63 (7)	73.8 (7)	0.07 (9)	69.1 (7)	0.11 (11)	78.6 (8)	0.30 (10)
HDDM-W	88.4 (5)	0.65 (6)	89.5 (6)	0.72 (6)	75.9 (5)	0.67 (6)	73.9 (6)	0.53 (6)	69.7 (6)	0.58 (6)	79.5 (4)	0.63 (6)
Page-Hinkley	88.2 (7)	0.65 (6)	86.0 (11)	0.06 (11)	74.8 (8)	0.52 (10)	73.5 (9)	0.07 (9)	68.5 (10)	0.11 (11)	78.2 (10)	0.28 (12)
KSWIN	88.0 (9)	0.42 (9)	88.3 (9)	0.66 (9)	74.5 (11)	0.53 (9)	73.7 (8)	0.63 (4)	68.9 (8)	0.43 (8)	78.7 (7)	0.53 (7)
MDDM	89.1 (4)	0.67 (5)	90.3 (5)	0.75 (5)	75.5 (6)	0.75 (4)	73.8 (7)	0.60 (5)	69.8 (5)	0.61 (5)	78.4 (9)	0.68 (5)
ADDM	89.8 (3)	0.86 (3)	91.9 (4)	0.77 (4)	76.6 (4)	0.72 (5)	75.0 (4)	0.69 (3)	71.3 (3)	0.77 (3)	79.3 (5)	0.76 (4)
PUDD	89.8 (3)	0.83 (4)	92.6 (2)	0.83 (3)	77.1 (3)	0.76 (3)	76.4 (2)	0.74 (2)	71.9 (2)	0.74 (4)	79.7 (3)	0.78 (3)
Meta-ADD	90.9 (2)	0.91 (2)	92.5 (3)	0.92 (2)	77.1 (3)	0.81 (2)	**84.8 (1)**	0.74 (2)	**80.4 (1)**	0.79 (2)	81.4 (2)	0.83 (2)
AutoDD	**96.7 (1)**	**0.93 (1)**	**98.4 (1)**	**0.93 (1)**	**82.4 (1)**	**0.90 (1)**	75.8 (3)	**0.84 (1)**	70.9 (4)	**0.88 (1)**	**84.0 (1)**	**0.87 (1)**

TABLE V: Real-world datasets – GaussianNB. Rank in (·); **bold** = best, underline = second-best per column. DN rank: lower = better.

Method	Elec		Weather		Spam		Airlines		Poker		Mean	
	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN
No Detector	73.2 (13)	**0 (1)**	68.9 (12)	**0 (1)**	77.2 (12)	**0 (1)**	59.3 (13)	**0 (1)**	54.6 (6)	**0 (1)**	66.6 (13)	**0 (1)**
ADWIN	79.2 (11)	22 (3)	69.3 (11)	7 (4)	92.1 (9)	2 (3)	62.5 (7)	76 (5)	54.3 (7)	3 (2)	71.5 (10)	22 (4)
DDM	82.3 (8)	22 (3)	70.6 (9)	5 (2)	91.8 (11)	2 (3)	61.8 (9)	20 (2)	54.2 (8)	3 (2)	72.1 (8)	10.4 (2)
EDDM	82.4 (7)	22 (3)	73.8 (6)	9 (5)	92.9 (5)	2 (3)	61.3 (11)	182 (11)	53.6 (10)	4 (3)	72.8 (7)	43.8 (6)
HDDM-A	82.7 (6)	22 (3)	72.2 (8)	9 (5)	92.5 (8)	2 (3)	62.6 (6)	114 (7)	54.6 (6)	6 (4)	72.9 (6)	30.6 (5)
HDDM-W	81.9 (9)	22 (3)	72.6 (7)	9 (5)	91.9 (10)	1 (2)	61.0 (12)	269 (12)	52.3 (11)	394 (12)	71.9 (9)	139 (13)
Page-Hinkley	78.0 (12)	16 (2)	69.4 (10)	6 (3)	91.8 (11)	1 (2)	62.8 (5)	64 (3)	54.3 (7)	17 (5)	71.3 (12)	20.8 (3)
KSWIN	81.1 (10)	22 (3)	68.0 (13)	9 (5)	92.6 (7)	2 (3)	61.6 (10)	269 (12)	53.7 (9)	266 (11)	71.4 (11)	113.6 (12)
MDDM	83.9 (5)	22 (3)	74.8 (5)	9 (5)	92.8 (6)	2 (3)	62.2 (8)	110 (6)	56.1 (4)	190 (10)	74.0 (5)	66.6 (9)
ADDM	84.8 (3)	22 (3)	76.3 (2)	9 (5)	93.9 (3)	2 (3)	63.9 (3)	74 (4)	56.5 (2)	177 (7)	75.1 (2)	56.8 (8)
PUDD	84.4 (4)	22 (3)	75.4 (3)	9 (5)	93.2 (4)	2 (3)	64.0 (2)	117 (8)	56.2 (3)	81 (6)	74.6 (4)	46.2 (7)
Meta-ADD	85.4 (2)	22 (3)	74.9 (4)	9 (5)	94.5 (2)	2 (3)	63.8 (4)	139 (10)	55.5 (5)	182 (8)	74.8 (3)	70.8 (11)
AutoDD	**89.6 (1)**	22 (3)	**80.0 (1)**	9 (5)	**97.9 (1)**	2 (3)	**68.2 (1)**	118 (9)	**59.7 (1)**	188 (9)	**79.1 (1)**	67.8 (10)

TABLE VI: Real-world datasets – HAT. Rank in (·); **bold** = best, underline = second-best per column. DN rank: lower = better.

Method	Elec		Weather		Spam		Airlines		Poker		Mean	
	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN
No Detector	76.5 (13)	**0 (1)**	71.4 (12)	**0 (1)**	80.1 (12)	**0 (1)**	62.1 (12)	**0 (1)**	56.9 (11)	**0 (1)**	69.4 (12)	**0 (1)**
ADWIN	82.4 (11)	22 (3)	73.1 (10)	9 (4)	94.6 (8)	2 (3)	66.3 (5)	76 (5)	57.7 (8)	3 (2)	74.8 (9)	22.4 (4)
DDM	85.5 (7)	22 (3)	73.5 (9)	7 (3)	93.9 (11)	2 (3)	65.2 (9)	20 (2)	58.0 (6)	3 (2)	75.2 (8)	10.8 (2)
EDDM	85.2 (8)	22 (3)	77.1 (6)	9 (4)	94.8 (6)	2 (3)	64.6 (11)	182 (11)	57.8 (7)	4 (3)	75.9 (6)	43.8 (6)
HDDM-A	86.3 (6)	22 (3)	75.8 (8)	9 (4)	94.7 (7)	2 (3)	66.1 (6)	114 (7)	58.5 (5)	6 (4)	76.3 (5)	30.6 (5)
HDDM-W	85.1 (9)	22 (3)	76.0 (7)	9 (4)	94.5 (9)	1 (2)	64.9 (10)	269 (12)	56.8 (12)	404 (12)	75.5 (7)	141 (13)
Page-Hinkley	81.7 (12)	16 (2)	72.8 (11)	6 (2)	94.2 (10)	1 (2)	66.5 (4)	64 (3)	57.5 (9)	17 (5)	74.5 (11)	20.8 (3)
KSWIN	84.3 (10)	22 (3)	71.2 (13)	9 (4)	94.9 (5)	2 (3)	65.4 (8)	269 (12)	57.0 (10)	266 (11)	74.6 (10)	113.6 (12)
MDDM	86.5 (5)	22 (3)	78.0 (5)	9 (4)	94.6 (8)	2 (3)	66.0 (7)	110 (6)	60.0 (2)	190 (10)	77.0 (4)	66.6 (9)
ADDM	87.3 (4)	22 (3)	79.2 (2)	9 (4)	95.8 (3)	2 (3)	66.9 (3)	74 (4)	59.6 (4)	177 (7)	77.8 (2)	56.8 (8)
PUDD	87.5 (3)	22 (3)	78.6 (3)	9 (4)	95.6 (4)	2 (3)	67.7 (2)	117 (8)	59.7 (3)	81 (6)	77.8 (2)	46.2 (7)
Meta-ADD	87.9 (2)	22 (3)	78.2 (4)	9 (4)	95.9 (2)	2 (3)	66.5 (4)	139 (10)	59.6 (4)	182 (8)	77.6 (3)	70.8 (11)
AutoDD	**93.8 (1)**	22 (3)	**85.1 (1)**	9 (4)	**99.0 (1)**	2 (3)	**72.6 (1)**	118 (9)	**65.1 (1)**	188 (9)	**83.1 (1)**	67.8 (10)



(a) SEA (GNB) (b) SEA (HAT) (c) Poker (GNB) (d) Poker (HAT)

Fig. 3: Prequential accuracy over time. SEA (synthetic) and Poker (real-world) datasets with GaussianNB and HAT base models.

A. Synthetic Datasets

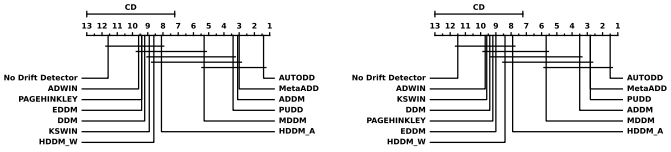
Tables III and IV compare all methods on synthetic benchmarks with known drift points. AutoDD achieves the best mean accuracy and F1 under both base models, with Meta-ADD ranking second across most datasets. The advantage is largest on datasets with abrupt drift, where adaptive detector switching is most beneficial, and narrows on gradual drift where Meta-ADD's learned representations are competitive. Fixed detectors exhibit inconsistent rankings across drift types, motivating the adaptive selection approach (Figures 3a–3b). Per-drift-type results are in Appendix A (Table VIII).

B. Real-world Datasets

Tables V and VI report results on real-world benchmarks where drift patterns are unknown. AutoDD ranks first on mean accuracy under both base models, with the improvement consistent across all five datasets. Detection counts remain within the consensus range of the compared methods, avoiding both extremes of over- and under-detection. Figures 3c and 3d confirm that AutoDD tracks or leads the baselines on the Poker stream and recovers promptly after accuracy drops.

C. Statistical Significance

Figures 4a and 4b show Critical Difference diagrams from the Friedman–Nemenyi test ($\alpha = 0.05$). AutoDD achieves the best mean rank under both base models, and the No Detector baseline ranks last, confirming that drift detection is beneficial across these benchmarks.



(a) GaussianNB (b) HAT

Fig. 4: Critical Difference diagrams (Friedman-Nemenyi, $\alpha = 0.05$). AutoDD achieves the best average rank under both base models.

D. Ablation Study

Table VII reports performance when each component is removed in turn, using GaussianNB averaged across all datasets. Removing the Level 1 LinUCB detector selector causes the largest accuracy drop, confirming that detector-stream matching is the primary driver of adaptive performance. Removing Level 2 Thompson Sampling produces a smaller but meaningful degradation, and all three meta-feature categories provide complementary gains, with statistical features contributing most. Figure 5 illustrates the resulting behaviour on the Hyperplane stream: the LinUCB layer continuously re-selects detectors as meta-features evolve, keeping the windowed error bounded and enabling rapid recovery after detected drifts.

TABLE VII: Ablation study results (accuracy averaged across all ten datasets; F1 over the five synthetic streams with known drift points).

Configuration	Accuracy	F1	ΔAcc
Full AutoDD	**80.5%**	**0.64**	–
w/o Temporal	74.2%	0.59	-6.3%
w/o Statistical	71.2%	0.57	-9.3%
w/o Stationarity	73.5%	0.59	-7.0%
w/o LinUCB (Level 1)	62.5%	0.50	-18.0%
w/o Thompson (Level 2)	75.3%	0.60	-5.2%

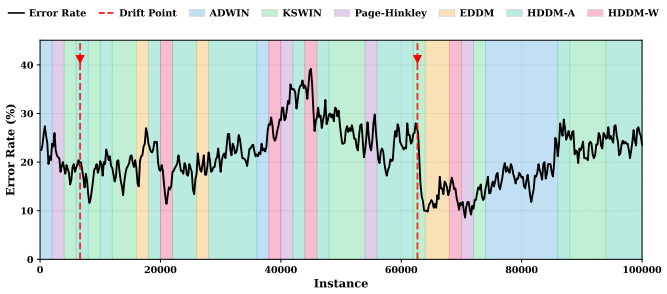
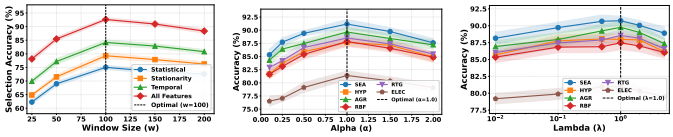


Fig. 5: Detector selection by AutoDD on the Hyperplane stream. Coloured bands show the dominant LinUCB-selected detector per block, the black curve the windowed prequential error rate, and dashed red lines the flagged drifts.

E. Sensitivity Study

Figure 6 shows AutoDD's sensitivity to its three key hyperparameters. Performance peaks at $w = 100$, where smaller windows introduce high variance and larger windows delay drift response. The exploration parameter α_{ucb} shows a clear optimum, with under-exploration causing exploitation bias and over-exploration reducing accuracy. The regularisation parameter λ_{reg} is robust across the tested range, and AutoDD degrades gracefully around the default settings.

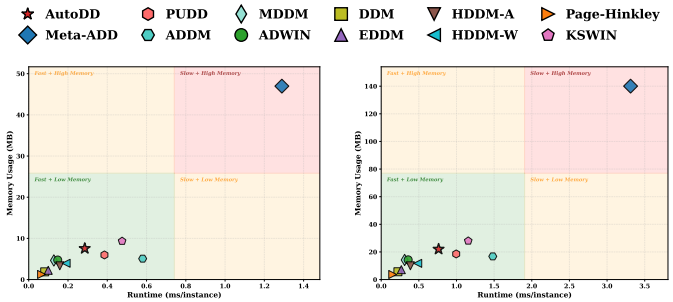


(a) w (b) α_{ucb} (c) λ_{reg}

Fig. 6: Hyperparameter sensitivity analysis showing selection accuracy across different values of (a) window size w, (b) exploration parameter α_{ucb}, and (c) regularisation parameter λ_{reg}.

F. Computational Efficiency

Figures 7a and 7b place all methods in the runtime-memory plane. AutoDD occupies the fast, low-memory region under both base models, with the hierarchical selection mechanism introducing only modest overhead. It is strictly more efficient than Meta-ADD and PUDD while delivering better detection performance, making it practical for high-velocity streams.



(a) GaussianNB (b) HAT

Fig. 7: Runtime vs. memory trade-off. AutoDD lies in the Fast + Low Memory quadrant under both base models.

VII. Discussion and Limitations

We highlight several boundaries of the current design. First, AutoDD automates detector and hyperparameter selection but not the sliding-window size, which the sensitivity analysis shows to be the most influential setting; the default value is robust across our benchmarks, yet selecting it for an unseen stream remains a manual step that an outer adaptive loop could close. Second, the reward is an accuracy-based proxy and therefore entangles detector quality with the downstream learner; we mitigate this through window-level smoothing and the significance threshold, and empirical results confirm that selection preferences are consistent across base learners, though a fully label-free reward remains an open direction. Third, to isolate detection from model selection we fix the base learner to GaussianNB and HAT, which are two standard but limited families, and adapt via full model reset, while richer learners and softer adaptation strategies (warm-starting, selective parameter resets, replay, ensemble updating) are left to future work. Finally, AutoDD targets tabular data streams, the prevalent setting in the drift-detection literature; non-tabular modalities such as image, audio, and text are outside the scope of this work [35].

VIII. Conclusion and Future Work

AutoDD automatically selects and configures drift detectors via a two-level hierarchical bandit using LinUCB for detector selection and Thompson Sampling for hyperparameter configuration. Experiments on synthetic and real-world datasets show consistent accuracy improvements over fixed and adaptive baselines without expert intervention. Future work includes automating window-size selection and extending selection to jointly optimise accuracy, detection latency, and computational efficiency.

References

[1] J. ao Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys (CSUR)*, vol. 46, no. 4, pp. 1–37, 2014.

[2] A. Bifet, G. Holmes, B. Pfahringer, P. Kranen, H. Kremer, T. Jansen, and T. Seidl, "MOA: Massive online analysis, a framework for stream classification and clustering," in *Proceedings of the First Workshop on Applications of Pattern Analysis*. PMLR, 2010, pp. 44–50.

[3] V. Losing, B. Hammer, and H. Wersing, "Incremental on-line learning: A review and comparison of state of the art algorithms," *Neurocomputing*, vol. 275, pp. 1261–1274, 2018.

[4] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.

[5] F. Hutter, L. Kotthoff, and J. Vanschoren, "Automated machine learning: Methods, systems, challenges," in *Automated Machine Learning*. Springer, 2019.

[6] B. Celik, P. Singh, and J. Vanschoren, "Online AutoML: An adaptive AutoML framework for online learning," *Machine Learning*, vol. 112, no. 6, pp. 1897–1921, 2023.

[7] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "ASML: A scalable and efficient AutoML solution for data streams," in *Proceedings of the Third International Conference on Automated Machine Learning (AutoML)*, ser. Proceedings of Machine Learning Research. PMLR, 2024.

[8] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proceedings of the 2007 SIAM International Conference on Data Mining*. SIAM, 2007, pp. 443–448.

[9] H. Yu, J. Lu, and G. Zhang, "Meta-ADD: A meta-learning based pre-trained model for concept drift active detection," *Information Sciences*, vol. 578, pp. 1–14, 2021.

[10] R. S. M. d. Barros and S. G. T. d. C. d. Santos, "A large-scale comparison of concept drift detectors," *Information Sciences*, vol. 451–452, pp. 348–370, 2018.

[11] C. Alippi, G. Boracchi, and M. Roveri, "Hierarchical change-detection tests," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 2, pp. 246–258, 2017.

[12] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proceedings of the 19th International Conference on World Wide Web (WWW)*. ACM, 2010, pp. 661–670.

[13] S. Agrawal and N. Goyal, "Analysis of Thompson sampling for the multi-armed bandit problem," in *Proceedings of the 25th Annual Conference on Learning Theory (COLT)*. PMLR, 2012, pp. 39.1–39.26.

[14] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Brazilian Symposium on Artificial Intelligence*. Springer, 2004, pp. 286–295.
[15] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavaldà, and R. Morales-Bueno, "Early drift detection method," in *Fourth International Workshop on Knowledge Discovery from Data Streams*, vol. 6, 2006, pp. 77–86.
[16] I. Friás-Blanco, J. de Campo-Ávila, G. Ramos-Jiménez, R. Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota, "Online and non-parametric drift detection methods based on Hoeffding's bounds," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 3, pp. 810–823, 2015.
[17] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
[18] A. Pesaranghader, H. Viktor, and E. Paquet, "McDiarmid drift detection methods for evolving data streams," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2017, pp. 1663–1670.
[19] M. Z. A. Mayaki and M. Riveill, "Autoregressive based drift detection method," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2022.
[20] R. S. Tsay, "Testing and modeling threshold autoregressive processes," *Journal of the American Statistical Association*, vol. 84, pp. 231–240, 1989. [Online]. Available: https://api.semanticscholar.org/CorpusID:17630406
[21] D. Kifer, S. Ben-David, and J. Gehrke, "Detecting change in data streams," in *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*, vol. 4, 2004, pp. 180–191.
[22] T. Dasu, S. Krishnan, S. Venkatasubramanian, and K. Yi, "An information-theoretic approach to detecting changes in multidimensional data streams," in *Proceedings of Symposium on the Interface of Statistics, Computing Science, and Applications*, 2006.
[23] N. Lu, G. Zhang, and J. Lu, "Concept drift detection via competence models," *Artificial Intelligence*, vol. 209, pp. 11–28, 2014.
[24] C. Raab, M. Heusinger, and F.-M. Schleif, "Reactive soft prototype computing for concept drift streams," *Neurocomputing*, vol. 416, pp. 340–351, 2020.
[25] D. M. dos Reis, P. Flach, S. Matwin, and G. Batista, "Fast unsupervised online drift detection using incremental Kolmogorov-Smirnov test," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 1545–1554.
[26] P. Lu, J. Lu, A. Liu, and G. Zhang, "Early concept drift detection via prediction uncertainty," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025.
[27] C. Kulbach, J. Montiel, M. Bahri, M. Heyden, and A. Bifet, "Evolution-based online automated machine learning," in *Advances in Knowledge Discovery and Data Mining (PAKDD)*, ser. Lecture Notes in Computer Science, vol. 13280. Springer, Cham, 2022, pp. 472–484.
[28] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári, "Improved algorithms for linear stochastic bandits," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011, pp. 2312–2320.
[29] T. Lattimore and C. Szepesvári, *Bandit Algorithms*. Cambridge University Press, 2020.
[30] V. Dani, T. P. Hayes, and S. M. Kakade, "Stochastic linear optimization under bandit feedback," in *Proceedings of the 21st Annual Conference on Learning Theory*, 2008, pp. 355–366.
[31] S. Agrawal and N. Goyal, "Further optimal regret bounds for Thompson sampling," in *Proceedings of the 16th International Conference on Artificial Intelligence and Statistics*. PMLR, 2013, pp. 99–107.
[32] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM Press, 2000, pp. 71–80.
[33] V. M. A. Souza, D. M. dos Reis, A. G. Maletzke, and G. E. A. P. A. Batista, "Challenges in benchmarking stream learning algorithms with real-world data," *Data Mining and Knowledge Discovery*, vol. 34, no. 6, pp. 1805–1858, 2020.
[34] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem, and A. Bifet, "River: Machine learning for streaming data in Python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.
[35] V. M. A. Souza, A. R. S. Parmezan, F. A. Chowdhury, and A. Mueen, "Efficient unsupervised drift detector for fast and high-dimensional data streams," *Knowledge and Information Systems*, vol. 63, no. 6, pp. 1497–1527, 2021.

APPENDIX A
ADDITIONAL RESULTS

Figure 8 plots mean detector selection frequency under GaussianNB against HAT across all datasets. All detectors lie near the diagonal (Spearman $\rho = 0.96$, $p < 0.001$), confirming the proxy tracks detector quality rather than base-learner artefacts.

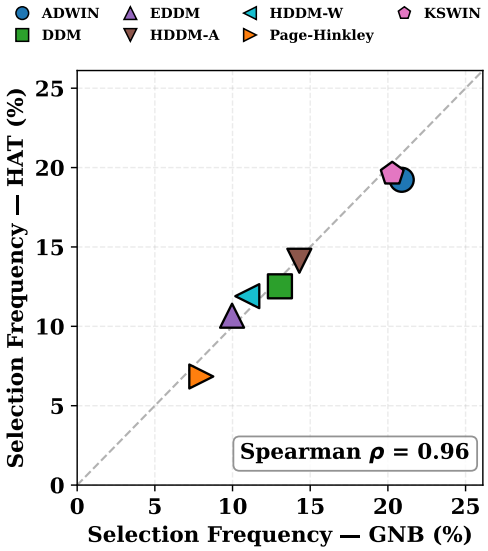


Fig. 8: Selection frequency per detector under GaussianNB vs. HAT.

A. Per-Drift-Type Performance

Table VIII breaks down accuracy by drift type. AutoDD leads on sudden, incremental, and recurring drift, while Meta-ADD edges ahead on gradual drift owing to its learned drift-type representations.

TABLE VIII: Accuracy (%) by drift type (GaussianNB).

Method	Sudden	Gradual	Incremental	Recurring
ADWIN	82.4	71.3	68.5	75.2
DDM	79.8	74.6	72.1	71.8
EDDM	75.2	78.4	74.8	69.3
HDDM-A	81.7	72.1	69.8	74.5
HDDM-W	80.1	76.2	73.5	74.6
Page-Hinkley	81.9	70.8	68.2	75.8
KSWIN	76.5	77.8	75.2	72.1
MDDM	81.3	75.8	74.1	73.9
ADDM	82.9	77.2	75.6	74.8
PUDD	82.1	79.5	76.8	75.4
Meta-ADD	84.6	**82.3**	78.9	77.8
AutoDD	**88.2**	81.7	**80.4**	**79.6**

B. Detector Selection Patterns

Table IX shows that AutoDD's learned selection policy aligns with domain knowledge: ADWIN and DDM are preferred for high-velocity streams, while KSWIN dominates for high-noise and high-dimensional streams.

TABLE IX: Detector selection frequency (%) by stream characteristic.

Detector	High Velocity	High Noise	High Dim.
ADWIN	28.4	12.3	15.7
DDM	22.1	8.5	11.2
EDDM	8.3	18.7	14.5
HDDM-A	15.2	10.4	12.8
HDDM-W	12.6	14.2	13.1
Page-Hinkley	5.8	8.1	7.2
KSWIN	7.6	27.8	25.5

C. Detection Delay

Table X reports mean detection delay on synthetic datasets. AutoDD achieves the lowest mean delay across all comparators, consistent with its tendency to select responsive detectors during drift-prone windows while avoiding oversensitive choices during stable periods.

TABLE X: Mean detection delay (instances) on synthetic datasets. Lower is better.

Method	SEA	HYP	AGR	RBF	Mean
ADWIN	45.2	78.3	52.1	81.4	64.3
DDM	38.7	62.5	48.9	65.2	53.8
EDDM	52.3	71.8	61.4	74.6	65.0
HDDM-A	43.8	75.1	50.3	78.6	61.9
HDDM-W	41.5	58.2	45.7	62.8	52.1
Page-Hinkley	46.9	80.5	53.8	83.2	66.1
KSWIN	49.1	68.7	58.2	71.4	61.9
MDDM	39.4	55.8	44.2	60.5	50.0
ADDM	36.5	51.3	41.6	57.8	46.8
PUDD	32.8	45.2	37.5	51.6	41.8
Meta-ADD	35.2	48.6	39.8	55.3	44.7
AutoDD	**28.4**	**42.1**	**34.2**	**48.7**	**38.4**

D. Convergence Analysis

Table XI tracks proxy-oracle selection agreement across stream quarters. AutoDD's agreement rises monotonically from the first to the final quarter across all datasets, confirming that the LinUCB layer learns reliable detector selections from the reward signal as the stream progresses.

TABLE XI: AutoDD proxy-oracle selection accuracy (%) at different stream positions.

Dataset	0–25%	25–50%	50–75%	75–100%
SEA	71.2	85.4	91.8	94.3
HYP	68.5	82.1	89.6	93.1
AGR	65.8	79.3	87.2	91.5
Elec	72.4	84.7	90.3	92.8
Poker	69.1	81.6	88.9	93.6
Mean	69.4	82.6	89.6	93.1

APPENDIX B
META-FEATURE FORMULATIONS

This appendix gives the full mathematical formulation of each of the 15 online meta-features that form the context vector $\boldsymbol{\phi}_t$. Table XII lists, per category, the closed-form expression or online estimator used to compute every feature over the sliding window of size w.

TABLE XII: Meta-feature portfolio with mathematical formulations.

Category	Feature	Formula/Description
Statistical	Velocity	$v_t = n_t/\Delta t$
	Dimensionality	d
	Sparsity	$\frac{1}{wd}\sum_{i=1}^{w}\sum_{j=1}^{d}\mathbb{I}[\mathbf{x}_{i,j}=0]$
	Noise level	$H(\mathbf{X}) \approx -\frac{1}{w}\sum_{i=1}^{w}\log \hat{p}_k(\mathbf{x}_i)$ (k-NN, $k=5$)
	Class imbalance	$\max_c \lvert C_c\rvert / \min_c \lvert C_c\rvert$
Stationarity	Error volatility	$\sigma_{\text{err}}(t) = \sqrt{\frac{1}{w}\sum_{i=t-w}^{t}(e_i-\bar{e})^2}$
	KL divergence	$D_{\text{KL}}(P_t \Vert P_{t-1})$ (histogram, $\sqrt{w}$ bins)
	Autocorrelation	$\rho_t = \frac{\text{Cov}(e_t, e_{t-1})}{\sigma_{e_t}\sigma_{e_{t-1}}}$
	Mean shift	$\Delta\mu_t = \frac{1}{d}\sum_{j=1}^{d}\lvert\mu_{t,j}-\mu_{t-1,j}\rvert$
	Variance shift	$\Delta\sigma_t^2 = \frac{1}{d}\sum_{j=1}^{d}\lvert\sigma_{t,j}^2-\sigma_{t-1,j}^2\rvert$
Temporal	Trend	$\beta_{\text{trend}} = \frac{\text{Cov}(t,e_t)}{\text{Var}(t)}$
	Periodicity	$f_{\text{dom}} = \arg\max_f \lvert\mathcal{F}\{e_t\}(f)\rvert$
	Change frequency	$\frac{1}{\Delta t}\sum_{i=1}^{w}\mathbb{I}[\lvert\Delta e_i\rvert > \tau_{\Delta e}]$
	Change magnitude	$\frac{1}{\lvert\mathcal{C}_t\rvert}\sum_{i\in\mathcal{C}_t}\lvert\Delta e_i\rvert$
	Recurrence score	$\frac{1}{w^2}\sum_{i,j}\mathbb{I}[\lVert\mathbf{x}_i-\mathbf{x}_j\rVert < \epsilon]$